

**ICE Lab MATLAB Programming****7/1/2026****Rev. 1****ABSTRACT:**

This document describes the MATLAB Client Template (“ML_client_template.m”). This program is designed to work in unison with the LabVIEW Client Template project (“Client_Template_MODBUS.lvproj”) and enable the integration of advanced control strategies in the Instrumentation and Control Laboratory at Murdoch University as part of the School of Engineering and Energy. This document gives full coverage of the MATLAB Client Template and is intended to serve as an all-encompassing description of all programming function for the purposes of technical understanding, troubleshooting, and future development.

Contents

List of Figures.....	3
List of Tables.....	3
Document Control and Revision History	4
1.0 Purpose & Scope.....	5
2.0 Programming Philosophy	6
3.0 Programming	7
3.1 Using With LabVIEW Client Template.....	7
3.1.1 General Information	7
3.1.2 User Interface	9
3.2 Overall Programming Flow	10
3.3 Supporting Functions.....	14
3.3.1 “CommsCheck.m”	14
3.3.2 “CommsInit.m”	15
3.3.3 “Sequencing.m”	16
3.3.4 “SvrRead.m”	17
3.3.5 “SvrWrite.m”	18
APPENDIX A. Main Program “ML_client_template.m”	19
APPENDIX B. Function “commsCheck.m”	26
APPENDIX C. Function “commsInit.m”	28
APPENDIX D. Function “sequencing.m”	31
APPENDIX E. Function “svrRead.m”	33
APPENDIX F. Function “svrWrite.m”	35
APPENDIX G. 3 rd Party Program Interface (FULL)	37

List of Figures

Figure 1: Portion of LabVIEW client template responsible for reading values from scratchpad holding registers. MATLAB writes to associated registers (8 registers starting at offset +14 from panel start register address).....	8
Figure 2: Portion of LabVIEW client template responsible for writing values to scratchpad holding registers. MATLAB reads from associated registers (8 registers starting at offset +24 from panel start register address).	9
Figure 3: The LabVIEW Client Template front panel, partitioned sections correspond to: i. ES analog inputs/outputs, ii. online plotting of back panel configured numeric variables, iii. ES digital inputs/outputs, iv. ES panel communication and program telemetry, v. online system logging, vi. scratchpad monitoring ().	10
Figure 4: Portion of LabVIEW Client Template front panel for scratchpad interface monitoring.	10
Figure 5: The overall flow of LabVIEW main client template diagram. When programming control strategies in MATLAB LabVIEW acts as user interface for real-time process commands, e.g., changing setpoints or enabling different forms of control. <i>Note: Diagram is simplified, and only broad steps are presented.</i>	13
Figure 6: The overall flow of MATLAB supporting template program. MATLAB program requires parallel execution of LabVIEW main client template program and is intended to run continuously without user interference with all required user commands/inputs managed by LabVIEW main client template program. <i>Note: Diagram is simplified, and only broad steps are presented.</i>	14
Figure APX 1: Portion of client template main loop responsible for coordination of communication between LabVIEW and 3 rd party applications for the purposes of offloading complex control strategies.	37

List of Tables

Table 1: Document Control and Revision History.....	4
---	---

Document Control and Revision History

Table 1: Document Control and Revision History

Revision	Date	Author(s)	Reviewed by	Comments/Details
1	2026	Nicholas O’Keeffe		Initial Commit.

1.0 Purpose & Scope

The purpose of this document, as stated in the abstract, is to give full coverage of the integration of MATLAB, enabling comparatively easy integration of advanced control integration in the Murdoch university Instrumentation and Control Engineering (ICE) Laboratory. Further, it is intended to serve as an all-encompassing description of all programming function for the purposes of technical understanding, troubleshooting, and future development.

In proceeding sections, the full programming philosophy, working, and supporting functions are described. This aim is to give the intent of the program design, as well as birds-eye down to granular descriptions of the program flow and function.

Excluded from the scope of this document is:

- Full description of the **LabVIEW Client Template Project** – The LabVIEW Client Template Project is intended to serve as the backbone of client-side communication enabling full use of engineering stations (ES) ES1-ES8 in the Murdoch university Instrumentation and Control Engineering (ICE) Laboratory. Only portions of the project relevant to the use of the MATLAB template package are covered, and the entirety of the project is covered in a separate document, “ICE-1001 ICE Lab LabVIEW Programming”, which is supplied within the client template package.
- **MODBUS/TCP Server Design and Support** – Server-side programming and communication, and hardware level process integration are not covered here. This is covered in a separate vendor supplied document. At the time of the production of REV 1 of this document, this was titled, “ICE_Lab_User_Manual_REV2_2”. A copy of this is supplied within the client template package, however, care should be taken to ensure this is up to date, as work on ICE lab upgrades had not been finalised during the production of client template support documentation.

2.0 Programming Philosophy

The Murdoch university ICE lab is used for the practical experimentation of the theory-side of the Industrial Automation and Control Engineering major as part of the school of engineering and energy. While the lab requires programming for control systems integration with physical processes, the emphasis is on theoretical control systems. Ideally programming requirements for experimentation are to be minimised for this process.

Previously, the lab utilised LabVIEW for full-stack integration of physical hardware with control design capable student designed programming. This was intended to allow ongoing modification and upgrades of facility supporting software to be carried out by students, who over the course of relevant engineering majors, spent significant time working with LabVIEW. This philosophy had flaws, particularly surrounding reliability and system robustness.

As hardware reached end of life. An increase in student numbers and growing concerns regarding reliability prompted the upgrade of process integrating hardware and the change of operating philosophy to limit student influence on critical systems. With this change, the requirements for student operable programming across the whole system was no longer an issue, and more practical solutions could be employed. This change in philosophy led to the new server design which integrates the existing 4-20mA interfacing hardware with ADAMs RTUs, each communicating with a MODBUS/TCP server hosted by a Linux machine.

Recent years have seen the rise of more complex control strategies taught/experimentally integrated across various units in the ICAE/ICSE major of undergraduate/masters degrees. To support this the MATLAB client template was developed. “ML_client_template.m” is intended to function as a means to ~easily program the kinds of complex control strategies that are too cumbersome to be integrated directly in LabVIEW (e.g.: MPC, and intelligent control). As user interfaces are laborious in MATLAB, the template is designed only to handle the execution of control algorithms, with the LabVIEW client template retaining the role of online monitoring and process management.

For ease-of-use the MATLAB main-program (Appendix A.) – and the student modified portion of the template – is kept as minimalistic as possible, and management tasks assigned to “hidden” functions (Appendix B-F.) are maximalised.

3.0 Programming

The complete package, designed to allow interface with the physical ICE lab engineering stations (ESs) is made of two components:

1. The LabVIEW client template program – “Client_Template_MOD.vi” – The primary user interface, which must be used regardless of optional interface with MATLAB supporting programming for advanced control strategies.
2. The MATLAB supporting program – “ML_client_template.m” – The MATLAB template designed to work with the LABVIEW user interface, optional use for cases when control integration in LabVIEW becomes cumbersome.

The MATLAB supporting programming is broken down into a single program and supporting functions. This section sequentially covers these items, and interfacing with LabVIEW client template:

1. **Interfacing with LabVIEW client template** – “Client_Template_MOD.vi” Front facing program control – acts as human machine interface – user configurable.
2. **Main Script** – “ML_client_template.m” – Main MATLAB program – houses user developed control strategies.
3. **Supporting MATLAB functions** – Supporting functions serve two purposes, first, to enable readability and modular program design, and second, to hide “complex” programming from less experienced users.

3.1 Using With LabVIEW Client Template

3.1.1 General Information

The MATLAB client template is intended only for executing control algorithms too cumbersome to be programmed in LabVIEW directly. On top of this, programming intelligible HMI's in MATLAB is comparatively laborious. As such the template is programmed such that the standard LabVIEW client template – “Client_Template_MOD.vi” – maintains its functionality as the front facing user interface for giving commands and interacting directly with the ESs and the MATLAB client template cannot directly direct commands at the ES field outputs.

The LabVIEW client template retains sole control over self-assigning ownership of a panel to the MODBUS server if the panel is unoccupied. “ML_client_template.m” performs a check of the MODBUS register corresponding to the gate (offset + 8) before performing any writes to the selected ES's address space.

To enable the transaction of data between the front facing LabVIEW user interface and the MATLAB template a total of 16 “scratchpad” MODBUS holding registers are assigned to each ES address space functioning as two 8x16-bit UINT registers for facilitating data transfer from

LabVIEW to MATLAB, and vice versa. The LabVIEW scratchpad inputs (Figure 1, Appendix G.) correspond to the holding registers offset +14, and are assigned values by MATLAB [“ML_client_template.m” (main-script), Appendix A. lines 90-97, “svrWrite.m” (main-script supporting function), Appendix F.], and the LabVIEW scratchpad outputs correspond to holding registers offset +24, are assigned values by LabVIEW (Figure 2), and read by MATLAB [“ML_client_template.m” (main-script), Appendix A., lines 64-78; “svrRead.m” (main-script supporting function), Appendix E., line 54]. LabVIEW scratchpad outputs can be altered by changing the reference of local variables, pictured in Figure 2.

The scratchpad holding registers used in the interface can be read-from and written to by any MODBUS/TCP compliant programming language/environment, thus similar templates could be made; by extension, MODBUS/TCP compliant programming languages/environments could be programmed to handle the full pipeline of interface and control. The important takeaway is that as the configured MATLAB interface allows interface based on the LabVIEW set local ID, **it cannot function if the LabVIEW client template is not also simultaneously running on the same local machine.**

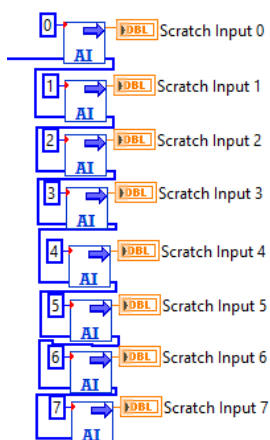


Figure 1: Portion of LabVIEW client template responsible for reading values from scratchpad holding registers. MATLAB writes to associated registers (8 registers starting at offset +14 from panel start register address).

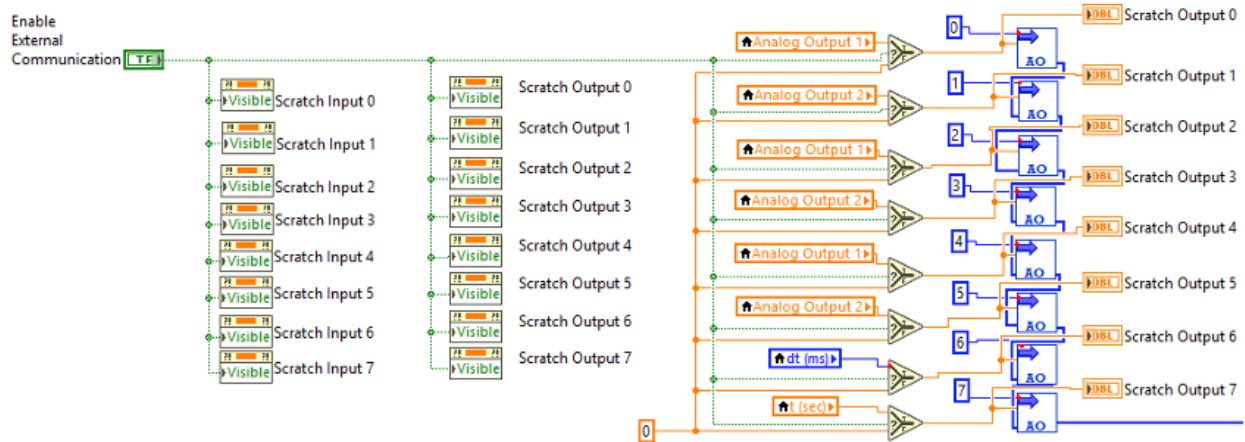


Figure 2: Portion of LabVIEW client template responsible for writing values to scratchpad holding registers. MATLAB reads from associated registers (8 registers starting at offset +24 from panel start register address).

3.1.2 User Interface

Figure 3 illustrates the LabVIEW client template user interface. Description of portions i.-v. are omitted from this document and are comprehensively discussed in “ICE-1001 ICE Lab LabVIEW Programming”. Portion vi. of Figure 3 enables scratchpad monitoring and limited control, continuously updating and displaying the values held in the holding registers associated with data transfer from MATLAB to LabVIEW (Offset +14), and data transfer from LabVIEW to MATLAB (Offset +24).

Figure 4 better illustrates portion vi. of Figure 3; “Enable External Communication” Boolean control intended to enable and disable system control routing from MATLAB exclusively on the LabVIEW side (i.e.: both applications maintain communication, user-defined programming in LabVIEW stops the execution of any commands), however, by default, this input variable only controls the visibility of scratchpad variable displays (When true → indicators visible, Figure 4 right, and when false → indicators invisible, Figure 4 left). Any actual influence of the “Enable External Communication” Boolean on the process requires user programming.

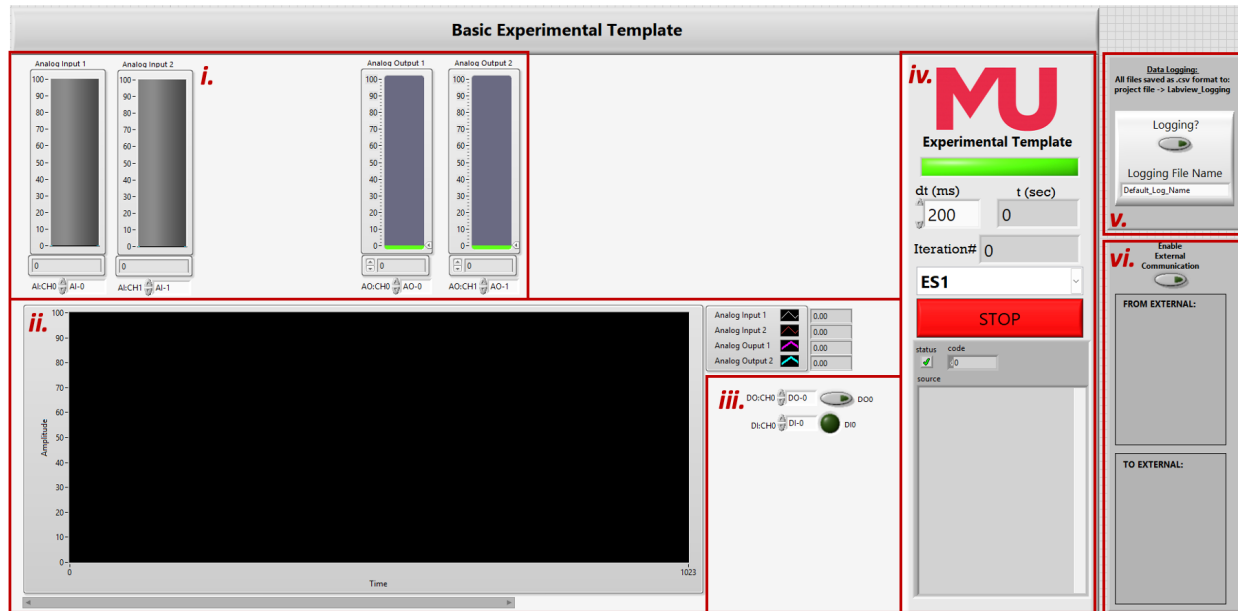


Figure 3: The LabVIEW Client Template front panel, partitioned sections correspond to: i. ES analog inputs/outputs, ii. online plotting of back panel configured numeric variables, iii. ES digital inputs/outputs, iv. ES panel communication and program telemetry, v. online system logging, vi. scratchpad monitoring ().

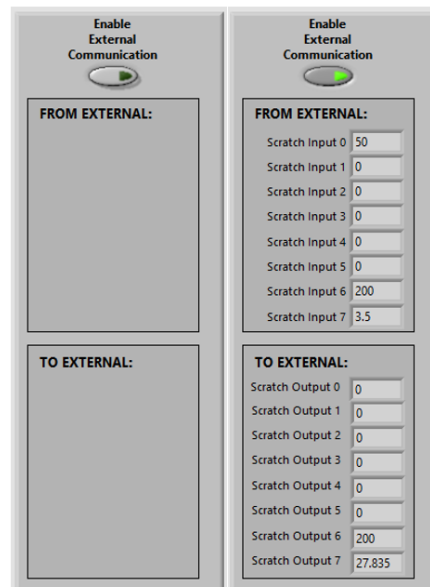


Figure 4: Portion of LabVIEW Client Template front panel for scratchpad interface monitoring.

3.2 Overall Programming Flow

This section will cover the main MATLAB script “ML_client_template.m”. It broadly covers each step of program execution to facilitate and interface with MODBUS/TCP registers relevant with monitoring of the physical process and any LabVIEW initiated commands, and the transfer of process commands to the primary LABVIEW script. Only broad steps will be covered, with all supporting function’s inputs, outputs, and working described in section 3.3. A flow diagram of

the primary LabVIEW script is given by Figure 5, illustrating the flow of relevant data from LabVIEW into MATLAB in the overall control pipeline, similarly a flow diagram of the main MATLAB script is given by Figure 6, illustrating the overall structure of the MATLAB client interface script. For completeness see script in appendix A or MATLAB script, both of which contain extensive commenting.

A. User Settings and Constant Initialisation

This section runs ONCE when the program starts, before the control loop and is used for two things, first setting the two required communication settings (Δt and ES number), and second, the declaration of any user required constants any controllers or processes within the script, e.g.: controller gains.

B. Initialise MODBUS Server Communication and Check Connection Conditions

Primarily, this step is responsible for checking that connection to the user specified ES is permitted. This is done by checking the server holding register associated with the panel gate. When a user connects to an ES with the primary LabVIEW script, if the panel is not locked by another user, a local-IP derived user ID is written to the gate register. MATLAB follows the same user ID generating procedure and compares this against the value read from the gate, if the user ID is confirmed to belong to the local machine, MATLAB communication is permitted, otherwise an error is returned, and the program is halted. In addition to confirming connection condition, this step also initialises some management variables associated with confirming connection and timekeeping.

C. Main Control Loop and Sequencing

The main control loop runs in perpetuity, only stopping either manually, via a stop command in MATLAB, or when the main LabVIEW client template is stopped and the local ID is cleared from the gate register. In this step the main loop is begun, and all sequencing is managed. The while loop runs freely, executing as fast as possible, at each execution of the loop the sequencing function checks the time since the last execution of all other contents of the while loop. When the time since the last execution is equal to or exceeds the user specified Δt , the loop contents is run.

D. Reading Registers From MODBUS/TCP Server

The “svrRead()” function is executed and input registers (associated with physical panel analog inputs), discrete inputs (associated with physical panel digital inputs), and holding registers associated with scratchpad variables written to by LabVIEW are read from the server. The values are all saved to a management variable (MGMT). After exiting the function, all relevant variables are moved to appropriately named variables to be used by the user in later programming.

E. Primary User Control Code

By default, this section is empty. This is the intended location for all custom user defined programming. Here the user can design control systems, or program logic and sequencing.

F. Writing Registers to MODBUS/TCP Server

Before writing values to the holding register associated with scratchpad variables read from by LabVIEW, user specified control commands must be assigned to appropriately named variables. These variables are then fed into the “svrWrite” function which to execute the write to server.

G. Program Telemetry

While the program is running values associated with control quality are tracked and displayed in the MATLAB command window:

1. For context, the time since MATLAB program start is displayed,
2. the average jitter – jitter is deviation in execution time from the desired sample time, e.g., for a desired Δt of 200ms, if the time since previous execution is 202ms, then the transmission jitter is 2ms,
3. Jitter 10% low, i.e., the average jitter of the worst 10% of measurements,
4. % jitter $\geq$ 5ms, i.e., the % of measurements with jitter greater than 5ms.

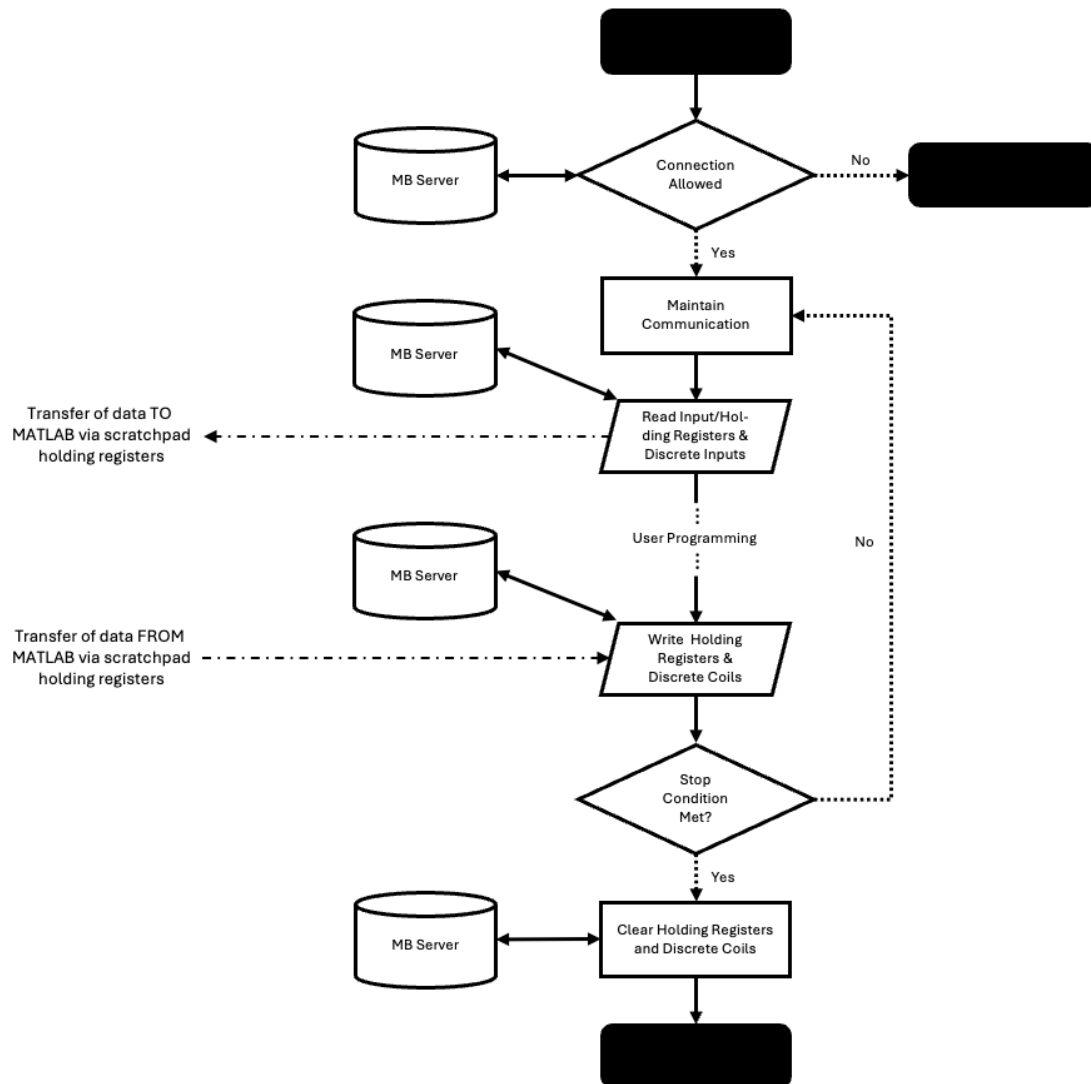


Figure 5: The overall flow of LabVIEW main client template diagram. When programming control strategies in MATLAB LabVIEW acts as user interface for real-time process commands, e.g., changing setpoints or enabling different forms of control. *Note: Diagram is simplified, and only broad steps are presented.*

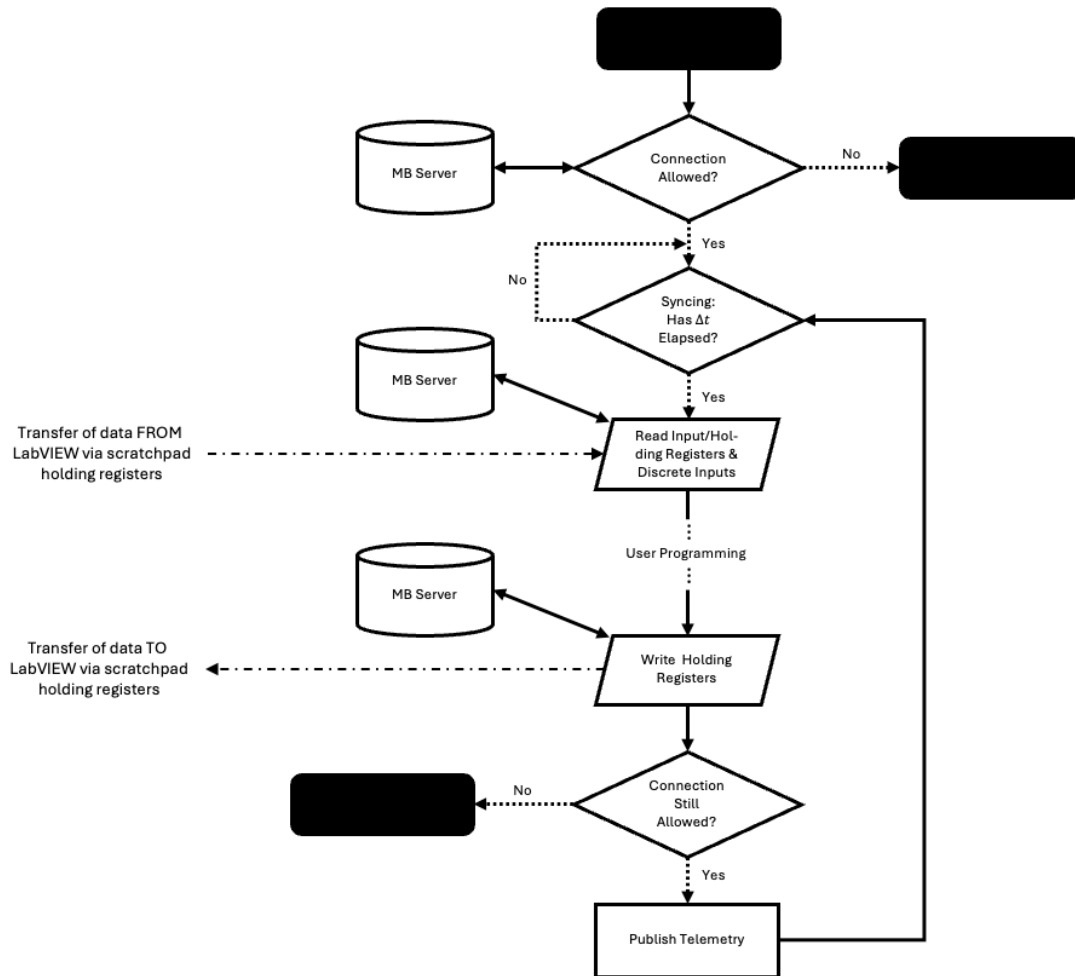


Figure 6: The overall flow of MATLAB supporting template program. MATLAB program requires parallel execution of LabVIEW main client template program and is intended to run continuously without user interference with all required user commands/inputs managed by LabVIEW main client template program. *Note: Diagram is simplified, and only broad steps are presented.*

3.3 Supporting Functions

This section describes the functionality and working of all custom MATLAB functions. For brevity, complete code is available in Appendices A through F.

3.3.1 “CommsCheck.m”

A. Function

Called once per control cycle (after writing outputs) to confirm that this machine is still the registered owner of its panel slot. If the LabVIEW supervisor has manually kicked this client, or the connection dropped and timed out, the gate register will no longer match this machine's ID and the program is halt.

B. Inputs

1. MGMT — Management structure (initialised by “commsInit” function) – Uses fields:
 - a. .srvr – MODBUS object (the live TCP/IP connection)
 - b. .OFFSET – register address offset for this panel’s block
 - c. .ID – unique 16-bit ID derived from 16-bit modulo of local machines IP address
 - d. .panel_number – user selected ES number

C. Outputs

1. None — The function either exits silently (all ok) or raises an error.

Also returns error/s:

- C03 — Gate register no longer matches this machine's ID, meaning this client has been removed from the panel by LabVIEW or a timeout.

D. Working

1. Read this machine's current IPv4 address from the OS
2. Re-derive the unique machine ID from the IP address
3. Read the panel gate register from the MODBUS server
4. Compare the gate register value to this machine's ID
5. If they do not match, raise Error C03 and halt the program

3.3.2 “CommsInit.m”

A. Function

Opens a TCP/IP connection to the MODBUS server, identifies this computer on the network using its IPv4 address, and checks that the requested panel slot is not already occupied by another user. If all checks pass, it initialises the MGMT management structure that is passed between all supporting functions throughout the session.

B. Inputs

1. panel_number — Integer (1, 2, 3 ...). Identifies which experimental panel this MATLAB instance is connecting to.
2. Δt — Desired control loop sample period in seconds.

C. Outputs

1. MGMT — Management structure containing:
 - a. .srvr – modbus object (the live TCP/IP connection)
 - b. .panel_number – copy of the panel number used
 - c. .OFFSET – register address offset for this panel's block
 - d. .ID – unique 16-bit integer ID derived from this machine's IP address
 - e. .gate – value read from the panel gate register

- f. `.dt` – requested sample period (seconds)
- g. `.init_time` – tic handle — counts total elapsed session time
- h. `.last_call` – tic handle — counts time since last cycle
- i. `.connected` – true once connection is confirmed

Also returns error/s

- C01 — Panel gate is held by a different machine ID (panel in use, or a previous session did not disconnect cleanly)
- C02 — No 192.168.x.x address found — machine is not on the lab network

D. Working

1. Create the MODBUS TCP/IP object and record the panel number
2. Calculate the register address offset for the selected panel
3. Read this machine's IPv4 address from the OS (via `ipconfig`)
4. Convert the IP address string into a unique 16-bit integer ID
5. Read the panel gate register from the server
6. Compare gate value to this machine's ID (panel-lock check)
7. Initialise timing variables and set `connected = true`
8. If not on lab network, raise Error C02

3.3.3 “Sequencing.m”

A. Function

Decides whether enough time has elapsed since the last control cycle to warrant running another one. Sets `MGMT.next_step = true` only when the measured elapsed time has exceeded the desired period (`dt`).

This is a non-blocking approach — unlike `pause()`, MATLAB never sits frozen waiting for a timer. The while-loop in the main program spins continuously, calling `sequencing()` every iteration. Actual control work only executes when `next_step` becomes true, which happens at $\sim \Delta t$ -second intervals.

B. Inputs

1. `MGMT` — Management structure. Fields read:
 - a. `.init_time` – tic handle for the total session clock (read only)
 - b. `.last_call` – tic handle for the per-cycle timer (reset when a new cycle is triggered)
 - c. `.dt` – desired sample period in seconds (read only)
 - d. `.t_real` – total elapsed time (seconds)
 - e. `.dt_real` – time since last cycle (seconds)

- f. `.next_step` – set true if a new control cycle should run, false otherwise

C. *Outputs*

2. MGMT — Management structure. Fields updated:
 - a. `.init_time` – tic handle for the total session clock (read only)
 - b. `.last_call` – tic handle for the per-cycle timer (reset when a new cycle is triggered)
 - c. `.dt` – desired sample period in seconds (read only)
 - d. `.t_real` – updated total elapsed time (seconds)
 - e. `.dt_real` – updated time since last cycle (seconds)
 - f. `.next_step` – set true if a new control cycle should run, false otherwise

D. *Working*

1. Measure total elapsed time since the program started
2. Measure elapsed time since the last control cycle completed
3. If elapsed time $\geq \Delta t$: flag a new cycle and reset the per-cycle timer
4. If elapsed time $< \Delta t$: flag no cycle this iteration

3.3.4 “SvrRead.m”

A. *Function*

Reads the latest values of all three input types from the MODBUS server each control cycle and stores them in the MGMT structure. The main program then unpacks them into the named variables:

- DI# – Discrete inputs 0-7, corresponding with ES digital inputs,
- AI# – Input registers 0-7, corresponding with ES analog inputs,
- MI# – Holding registers with offset +24, corresponding to scratchpad variable data transferred from LabVIEW to MATLAB.

It should be noted that MODBUS registers store 16-bit unsigned integers only. Floating-point values are therefore stored on the server scaled by $\times 100$ and rounded (e.g. a sensor reading of 1.23 is stored as the integer 123). Dividing by 100 here restores the original floating-point value.

B. *Inputs*

1. MGMT — Management structure input

C. *Outputs*

2. .MGMT — Management structure with updated fields:
 - a. `.dig_in` – [1 $\times$ 8 double] Digital input states (0 or 1)
 - b. `.in_reg` – [1 $\times$ 8 double] Analogue input values (scaled float)

- c. `.mi_reg` – [1×8 double] Move input values from LabVIEW (scaled float)

D. Working

1. Read 8 discrete input bits from offset +0 — panel digital inputs (e.g. switches)
2. Read 8 input registers from offset +0 — panel analogue inputs (e.g. sensors) and divide by 100 to restore float values
3. Read 8 holding registers from offset +24 — move inputs written by LabVIEW and divide by 100 to restore float values

3.3.5 “SvrWrite.m”

A. Function

Takes the eight output values (MO0–MO7) calculated by the student's control code, scales them to integers, and writes them to the MODBUS holding registers on the server. LabVIEW reads these registers each cycle and uses data according to user configured LabVIEW program.

It should be noted that MODBUS registers store 16-bit unsigned integers only. Floating-point outputs are therefore multiplied by 100 and rounded before writing (e.g. an output value of 1.23 is written to the register as 123).

B. Inputs

1. MGMT — Management structure input with function used subfields:
 - a. `.svr` -- modbus object (the live TCP/IP connection)
 - b. `.OFFSET` – register address offset for this panel's block
 - c. `.mo_reg` – storage for pre-written scratch pad variables to write to server
2. MO0–MO7 — Eight floating-point output values, one per allotted scratchpad holding register.

C. Outputs

1. MGMT — Updated structure with `.mo_reg` holding the scaled integer values – useful for debugging — inspect `MGMT.mo_reg` to see what was sent

D. Working

1. Scale each output float by ×100 and round to the nearest integer, storing the results in `MGMT.mo_reg`
2. Write all eight scaled values to the server in a single block write to ensure all outputs reach the server

APPENDIX A. Main Program “ML_client_template.m”

A direct copy and paste of full MATLAB program

```

1      clear, close all, clc
2      % =====
3      % ML_client_template.m - VERSION 1.0 - 2026
4      % MATLAB Client Template - Experimental Control Interface
5      % =====
6      %
7      % OVERVIEW:
8      % This is your starting-point template for writing a control strategy in
9      % MATLAB. It communicates with a MODBUS/TCP server running on the lab
10     % network that links this program to:
11     % - Physical field I/O on ES0-7 (sensors, actuators)
12     % - A LabVIEW user-interface panel that must be run simultaneously on
13     % the same physical machine to allow connection and real time commands.
14     %
15     % =====
16     % NOTE: FULL PROGRAM DESCRIPTION AT BOTTOM OF CODE
17     % =====
18     %
19     % WHERE TO WRITE YOUR CODE:
20     % >> Look at sections (Step #:...):
21     % > If proceeded by "MANAGEMENT" the section will be part of server
22     % communication and user modification is not required ...
23     % (FEEL FREE TO EDIT, WORST THAT WILL HAPPEN IS YOUR PROGRAM
24     % WON'T WORK)
25     % > If proceeded by "USER CONFIG." the section is intended for user
26     % interaction, either config./initialisation of program (STEP 1),
27     % or control-loop-execution/automation (STEP 5)
28     %
29     % SIGNAL NAMING CONVENTION:
30     % DI# - Digital Input : boolean 0 or 1 (e.g. limit switch, button)
31     % AI# - Analogue Input : scaled float value (e.g. sensor voltage)
32     % MI# - Move Input : value sent TO MATLAB from LabVIEW (e.g. setpoint)
33     % MO# - Move Output : value sent FROM MATLAB to LabVIEW (e.g. command)
34     % (# is a channel index 0-7, giving 8 channels of each type)
35     %
36     % =====
37
38     %% Step 1: ----- User settings and constant initialisation <<< USER CONFIG.

```

```

39     dt = 0.200; % Control loop sample period in SECONDS.
40
41     panel_number = 1; % The ES number your LabVIEW panel is using (1, 2, ..., 8).
42
43     % =====
44     % Define any required program constants here, e.g.: controller gains
45     % =====
46
47     %% Step 2: ----- Initialise MODBUS communication <<< MANAGEMENT
48     MGMT = commsInit(panel_number, dt);
49
50     % --- Telemetry initialisation (no editing needed) ---
51     telem_buf = []; % rolling buffer storing the last 100 measured cycle periods (s)
52     telem_chars = 0; % character count of the last telemetry line printed -
53     % used to overwrite command line each cycle to update telemetry
54
55     %% Step 3: ----- Main control loop (run until comms. loss) <<< CONTAINS USER CONFIG.
56     while MGMT.connected
57
58         % Step 3a: --- Check whether it is time for the next control cycle <<< MANAGEMENT
59         % sequencing() updates MGMT.next_step based on elapsed time vs dt.
60         MGMT = sequencing(MGMT);
61
62         if MGMT.next_step % True only once every dt seconds - run one control cycle
63             %% Step 4:---- Read all input registers from the MODBUS server <<< MANAGEMENT
64             MGMT = svrRead(MGMT);
65
66             % Unpack inputs into named variables for use in your control code.
67             % Read across each row: DI# = digital, AI# = analogue, MI# = from LabVIEW.
68             % Move inputs | Digital inputs (BOOL)| Analogue inputs (float)
69             % LabVIEW Scratch- | ES Digital input 0-7 | ES Analog input 0-7
70             % pad outputs 0-7 | |
71             MI0 = MGMT.mi_reg(1); DI0 = MGMT.dig_in(1); AI0 = MGMT.in_reg(1);
72             MI1 = MGMT.mi_reg(2); DI1 = MGMT.dig_in(2); AI1 = MGMT.in_reg(2);
73             MI2 = MGMT.mi_reg(3); DI2 = MGMT.dig_in(3); AI2 = MGMT.in_reg(3);
74             MI3 = MGMT.mi_reg(4); DI3 = MGMT.dig_in(4); AI3 = MGMT.in_reg(4);
75             MI4 = MGMT.mi_reg(5); DI4 = MGMT.dig_in(5); AI4 = MGMT.in_reg(5);
76             MI5 = MGMT.mi_reg(6); DI5 = MGMT.dig_in(6); AI5 = MGMT.in_reg(6);
77             MI6 = MGMT.mi_reg(7); DI6 = MGMT.dig_in(7); AI6 = MGMT.in_reg(7);
78             MI7 = MGMT.mi_reg(8); DI7 = MGMT.dig_in(8); AI7 = MGMT.in_reg(8);

```

```

79
80     %% Step 5: ----- User control program << USER CONFIG.
81
82     % =====
83     % WRITE YOUR CODE HERE
84     % =====
85
86     %% Step 6: - Assign output variables to send back to LabVIEW << USER CONFIG.
87     % DO NOT CHANGE VARIABLE NAMES M00-M07
88     % REPLACE VARIABLE NAMES MI0-MI7 WITH ANY VARIABLES YOU DEFINED
89     % ABOVE THAT YOU WISH TO WRITE BACK TO LABVIEW
90     M00 = MI0; % LabVIEW Scratch Input 0
91     M01 = MI1; % LabVIEW Scratch Input 1
92     M02 = MI2; % LabVIEW Scratch Input 2
93     M03 = MI3; % LabVIEW Scratch Input 3
94     M04 = MI4; % LabVIEW Scratch Input 4
95     M05 = MI5; % LabVIEW Scratch Input 5
96     M06 = MI6; % LabVIEW Scratch Input 6
97     M07 = MI7; % LabVIEW Scratch Input 7
98
99     %% Step 7: ----- Write outputs to server and verify connection <<< MANAGEMENT
100    MGMT = svrWrite(MGMT, M00, M01, M02, M03, M04, M05, M06, M07);
101    commsCheck(MGMT); % Halts safely if the connection has been lost
102
103    %% Step 8: ----- Loop telemetry <<< MANAGEMENT
104    % Append this cycle's measured period to the rolling buffer
105    telem_buf(end+1) = MGMT.dt_real;
106    if numel(telem_buf) > 100
107        telem_buf = telem_buf(end-99:end); % keep only the last 100 cycles
108    end
109
110    % Compute jitter statistics over the buffer
111    deviations = abs(telem_buf - dt); % absolute deviation from dt per cycle
112    avg_jitter = mean(deviations) * 1000; % ms - mean deviation
113
114    sorted_buf = sort(telem_buf);
115    idx_90 = min(numel(sorted_buf), ceil(0.9 * numel(sorted_buf))); % index of 90th percentile
116    high90_dt = sorted_buf(idx_90) * 1000; % ms - worst 10% of cycles
117
118    pct_bad = 100 * mean(deviations > 0.005); % % of cycles with >5 ms deviation

```

```

119
120     % Build a fixed-width string and overwrite the previous telemetry line.
121     % repmat('\b',...) sends backspace characters to erase the previous print.
122     telem_str = sprintf( ...
123         '[Telemetry] t: %8.2f s | Avg jitter: %6.2f ms | 90% high dt: %6.2f ms | >5ms bad: %5.1f%% (target: %4.0f
124         ms)', ...
125         MGMT.t_real, avg_jitter, high90_dt, pct_bad, dt * 1000);
126     fprintf('%s', repmat('\b', 1, telem_chars), telem_str);
127     telem_chars = numel(telem_str); % save length so next cycle can erase it
128
129
130 end % end while MGMT.connected
131
132 % =====
133 %% Full program description
134 % =====
135 % STEP 1 – USER SETTINGS AND CONSTANT INITIALISATION
136 %
137 % This section runs ONCE when the program starts, before the control loop.
138 % Use it for two things:
139 %
140 % a) The two required communication settings (dt and panel_number).
141 %
142 % b) Any constant values your control algorithm needs – things that are
143 % fixed for the whole run and do not change cycle-to-cycle.
144 % Defining them here (rather than inside the loop) is good practice:
145 % it keeps your tuning parameters in one easy-to-find place, and
146 % avoids MATLAB re-creating the variable every single cycle.
147 %
148 % EXAMPLES of constants to define here:
149 % Kp = 1.5; % Proportional gain
150 % Ki = 0.3; % Integral gain
151 % Kd = 0.05; % Derivative gain
152 % iSum = 0; % Integral accumulator – initialised to zero
153 % % (state variables like this also go here)
154 %
155 % NOTE: Variables that carry state between cycles (e.g. an integral sum,
156 % previous error for a derivative term) must also be initialised
157 % here so they exist on the first cycle of the loop.
158 %

```

```

159 % =====
160 % STEP 2 – CONNECT TO THE MODBUS SERVER
161 % <<< YOU DO NOT NEED TO EDIT THIS SECTION
162 %
163 % commsInit() does the following automatically:
164 % – Opens a TCP/IP connection to the MODBUS server at 192.168.1.200
165 % – Identifies this computer on the network using its IP address
166 % – Checks that nobody else is already using your panel
167 % – Returns the MGMT structure which holds all connection and timing info
168 %
169 % If the connection fails, a descriptive error message will be displayed
170 % in the Command Window explaining what went wrong and how to fix it.
171 %
172 % =====
173 % STEP 3 – MAIN CONTROL LOOP INIT. AND PER CYCLE SEQUENCING
174 % The while-loop runs continuously for as long as MGMT.connected is true.
175 %
176 % Inside the loop, sequencing() acts as a non-blocking timer:
177 % – It measures how much time has passed since the last cycle.
178 % – If >= dt seconds have elapsed, it sets MGMT.next_step = true.
179 % – Only then does your control code run for one cycle.
180 %
181 % This is better than using pause() because MATLAB never sits frozen –
182 % the timing is accurate and the connection can be monitored continuously.
183 %
184 % -----
185 % STEP 3a – Check whether it is time for the next control cycle –
186 % sequencing() updates MGMT.next_step based on elapsed time vs dt.
187 %
188 % =====
189 % STEP 4 – READ INPUTS FROM THE SERVER
190 % svrRead() fetches the latest values from the MODBUS server.
191 % The lines below unpack those values into easy-to-use named
192 % variables. You do not need to edit this section.
193 %
194 % =====
195 % STEP 5 – User control program
196 % This is the only section you need to fill in. It runs once every
197 % dt seconds. Use the input variables read above to compute output
198 % commands, then assign those commands to M00–M07 in Step 6.

```

```

199 %
200 % AVAILABLE INPUTS each cycle:
201 % DI0-DI7 Digital inputs from the rig (0 or 1)
202 % AI0-AI7 Analogue inputs from the rig (e.g. voltage, position)
203 % MI0-MI7 Values sent from LabVIEW (e.g. setpoints, modes)
204 %
205 % YOU MUST SET M00-M07 (in Step 6) – these are sent to LabVIEW
206 % and on to the physical actuators on the rig.
207 %
208 % EXAMPLE – simple proportional (P) controller on channel 0:
209 % Kp = 2.0; % proportional gain
210 % error = MI0 - AI0; % setpoint (from LV) minus measurement
211 % M00 = Kp * error; % proportional output command
212 %
213 % EXAMPLE – use a digital input as an enable switch:
214 % if DI0 == 1
215 % M00 = Kp * (MI0 - AI0); % controller active
216 % else
217 % M00 = 0; % output zero when switch is off
218 % end
219 %
220 % =====
221 % STEP 6 – SET OUTPUT VALUES
222 % Assign a value to each M0# (Move Output) variable below.
223 % Replace the right-hand side (currently MI#) with whatever your
224 % control algorithm calculated in Step 5.
225 %
226 % The default M0# = MI# is a simple pass-through – it echoes
227 % the LabVIEW input straight back as the output, which is safe
228 % to run but performs no control action.
229 %
230 % CHANNELS YOU DON'T USE: Leave them as M0# = MI# (pass-through).
231 %
232 % =====
233 % STEP 7 – WRITE OUTPUTS AND CHECK THE CONNECTION
234 % You do not need to edit anything here.
235 %
236 % svrWrite() scales the M0# values and sends them to the server.
237 % commsCheck() verifies this machine is still authorised. If the
238 % supervisor has ended the session or a dropout occurred, the

```


25

APPENDIX B. Function “commsCheck.m”

A direct copy of full MATLAB function. Function responsible for checking status of MODBUS server connection, if LabVIEW user interface connects from panel, this function will halt MATLAB communication and return error.

```

1  function commsCheck(MGMT)
2  % =====
3  % commsCheck – Verify this client is still authorised on the server
4  % =====
5  %
6  % PURPOSE:
7  % Called once per control cycle (after writing outputs) to confirm that
8  % this machine is still the registered owner of its panel slot.
9  % If the LabVIEW supervisor has manually kicked this client, or the
10 % connection dropped and timed out, the gate register will no longer
11 % match this machine's ID and the program is halted safely.
12 %
13 % INPUT:
14 % MGMT – Management structure (initialised by commsInit).
15 % Uses fields: .srvr, .OFFSET, .ID, .panel_number
16 %
17 % OUTPUT:
18 % None – The function either exits silently (all ok) or raises an error.
19 %
20 % ERROR RAISED:
21 % C03 – Gate register no longer matches this machine's ID, meaning this
22 % client has been removed from the panel by LabVIEW or a timeout.
23 %
24 % STEPS:
25 % 1. Read this machine's current IPv4 address from the OS
26 % 2. Re-derive the unique machine ID from the IP address
27 % 3. Read the panel gate register from the MODBUS server
28 % 4. Compare the gate register value to this machine's ID
29 % 5. If they do not match, raise Error C03 and halt the program
30 %
31 % =====
32
33 % --- Step 1: Read this machine's current IPv4 address from the OS ---
34 % Same approach as commsInit – uses ipconfig to find the 192.168.x.x address.
35 [~, out] = system('ipconfig');
```

```
36
37 matches = regexp(out, 'IPv4 Address[^\:]*:\s*(192\.168\.d+\.d+)', 'tokens');
38
39 if ~isempty(matches)
40
41     % --- Step 2: Re-derive the unique machine ID from the current IP ---
42     % Recalculated from scratch each call rather than reading from MGMT.ID,
43     % so the check stays valid even if the network address changed mid-session.
44     id = matches{1}{1};
45     id = typecast(flipplr(uint8(str2num(strrep(id, '.', ' ')))), 'uint32');
46     MGMT.ID = mod(id, 65535);
47
48     % --- Step 3: Read the panel gate register from the server ---
49     % The gate register holds the ID of the currently authorised client.
50     % If this machine still owns the panel, the gate value equals MGMT.ID.
51     MGMT.gate = read(MGMT.srvr, 'holdingregs', MGMT.OFFSET + 8, 1);
52
53     % --- Steps 4 & 5: Compare gate to this machine's ID ---
54     % A mismatch means this client is no longer authorised. This can happen if:
55     % - The LabVIEW operator manually ended this client's session
56     % - The connection timed out and the panel was released to another user
57     if MGMT.gate ~= MGMT.ID
58         err_msg = append("Error C03:\n \tKicked from panel ES%d.\n", ...
59             append("\tLabview Basic Experimental Template may have halted locally, ", ...
60                 "\n\tor you may have been kicked manually from server."));
61         err_msg = sprintf(err_msg, MGMT.panel_number);
62         error(err_msg);
63     end
64
65 end
66
67 end
```

APPENDIX C. Function “commsInit.m”

A direct copy of full MATLAB function. Function responsible for checking availability of selected ES and blocking access if panel is in use.

```

1      function MGMT = commsInit(panel_number, dt)
2      % =====
3      % commsInit – Initialise MODBUS/TCP communication with the lab server
4      % =====
5      %
6      % PURPOSE:
7      % Opens a TCP/IP connection to the MODBUS server, identifies this
8      % computer on the network using its IPv4 address, and checks that the
9      % requested panel slot is not already occupied by another user.
10     % If all checks pass, it initialises the MGMT management structure
11     % that is passed between all supporting functions throughout the session.
12     %
13     % INPUTS:
14     % panel_number – Integer (1, 2, 3 ...). Identifies which experimental
15     % panel/rig this MATLAB instance is connecting to.
16     % dt – Desired control loop sample period in seconds.
17     %
18     % OUTPUT:
19     % MGMT – Management structure containing:
20     % .srvr modbus object (the live TCP/IP connection)
21     % .panel_number copy of the panel number used
22     % .OFFSET register address offset for this panel's block
23     % .ID unique 16-bit integer ID derived from this
24     % machine's IP address
25     % .gate value read from the panel gate register
26     % .dt requested sample period (seconds)
27     % .init_time tic handle – counts total elapsed session time
28     % .last_call tic handle – counts time since last cycle
29     % .connected true once connection is confirmed
30     %
31     % ERRORS RAISED:
32     % C01 – Panel gate is held by a different machine ID (panel in use, or
33     % a previous session did not disconnect cleanly)
34     % C02 – No 192.168.x.x address found – machine is not on the lab network
35     %
36     % STEPS:
37     % 1. Create the MODBUS TCP/IP object and record the panel number

```

```

38 % 2. Calculate the register address offset for the selected panel
39 % 3. Read this machine's IPv4 address from the OS (via ipconfig)
40 % 4. Convert the IP address string into a unique 16-bit integer ID
41 % 5. Read the panel gate register from the server
42 % 6. Compare gate value to this machine's ID (panel-lock check)
43 % 7. Initialise timing variables and set connected = true
44 % 8. If not on lab network, raise Error C02
45 %
46 % =====
47
48 % --- Step 1: Create the MODBUS TCP/IP connection object ---
49 % The server is always at the fixed lab IP address 192.168.1.200.
50 % The modbus() object handles all low-level TCP/IP communication.
51 MGMT.srvr = modbus('tcpip', '192.168.1.200');
52 MGMT.panel_number = panel_number;
53
54 % --- Step 2: Calculate the register address offset for this panel ---
55 % Each panel occupies a block of 100 registers on the server.
56 % Panel 1 starts at register 1, panel 2 at register 101, panel 3 at 201, etc.
57 MGMT.OFFSET = (panel_number - 1) * 100 + 1;
58
59 % --- Step 3: Read this machine's IPv4 address from the OS ---
60 % The Windows command 'ipconfig' lists all network interface details.
61 % The regular expression extracts any address in the 192.168.x.x range,
62 % which is the expected address range for the lab network.
63 [~, out] = system('ipconfig');
64
65 matches = regexp(out, 'IPv4 Address[^\:]*:\s*(192\.168\.d+\.d+)', 'tokens');
66
67 if ~isempty(matches)
68     % A 192.168.x.x address was found - this machine is on the lab network.
69
70     % --- Step 4: Convert the IP address string to a unique 16-bit ID ---
71     % The IP string (e.g. "192.168.1.42") is split into four byte values,
72     % reinterpreted as a single uint32, then reduced to a 16-bit range
73     % using mod(). This produces a consistent, unique ID per machine.
74     id = matches{1}{1};
75     id = typecast(fliplr(uint8(str2num(strrep(id, '.', ' ')))), 'uint32');
76     MGMT.ID = mod(id, 65535);
77

```

```
78      % --- Step 5: Read the panel gate register from the server ---
79      % The gate register (at OFFSET + 8) stores the ID of the machine that
80      % currently owns this panel. A value of 0 means the panel is free.
81      MGMT.gate = read(MGMT.srvr, 'holdingregs', MGMT.OFFSET + 8, 1);
82
83      % --- Step 6: Panel-lock check ---
84      % If the gate register already holds a different ID, another session is
85      % active. Raise Error C01 to prevent two clients writing to the same panel.
86      % (If this is YOUR machine reconnecting after a crash, wait for the panel
87      % timeout or ask the supervisor to reset the panel from the server.)
88      if MGMT.gate ~= MGMT.ID
89          err_msg = append("Error C01:\n \tES%d locked by other user.\n", ...
90                          append("\tPanel may be in use, or a disconnection error has occurred.", ...
91                              "\n\tFor disconnection errors, reset panel from server or wait for panel timeout"));
92          err_msg = sprintf(err_msg, panel_number);
93          error(err_msg);
94      end
95
96      % --- Step 7: Initialise timing variables and confirm connection ---
97      MGMT.dt = dt; % Store desired sample period for use in sequencing()
98      MGMT.init_time = tic(); % Start total-elapsed-time counter (session clock)
99      MGMT.last_call = tic(); % Start per-cycle timer (resets each control cycle)
100     MGMT.connected = true; % Signal to the main loop that the connection is live
101
102     else
103         % --- Step 8: Not on the lab network – raise Error C02 ---
104         % ipconfig returned no 192.168.x.x address, so the MODBUS server cannot
105         % be reached. Check the network cable or Wi-Fi connection and try again.
106         err_msg = append("Error C02:\n \tMODBUS server not found.\n", ...
107                         append("\tCheck network interfaces and try again.));
108         err_msg = sprintf(err_msg);
109         error(err_msg);
110     end
111
112     end
```

APPENDIX D. Function “sequencing.m”

A direct copy of full MATLAB function. Function responsible tracking user selected sample time (dt).

```

1  function [MGMT] = sequencing(MGMT)
2  % =====
3  % sequencing – Non-blocking sample-rate timing for the control loop
4  % =====
5  %
6  % PURPOSE:
7  % Decides whether enough time has elapsed since the last control cycle
8  % to warrant running another one. Sets MGMT.next_step = true only when
9  % the measured elapsed time has exceeded the desired period (dt).
10 %
11 % This is a non-blocking approach – unlike pause(), MATLAB never sits
12 % frozen waiting for a timer. The while-loop in the main program spins
13 % continuously, calling sequencing() every iteration. Actual control
14 % work only executes when next_step becomes true, which happens at
15 % approximately dt-second intervals.
16 %
17 % INPUT / OUTPUT:
18 % MGMT – Management structure. Fields read and updated:
19 % .init_time tic handle for the total session clock (read only)
20 % .last_call tic handle for the per-cycle timer (reset when
21 % a new cycle is triggered)
22 % .dt desired sample period in seconds (read only)
23 % .t_real updated with total elapsed time (seconds)
24 % .dt_real updated with time since last cycle (seconds)
25 % .next_step set true if a new control cycle should run,
26 % false otherwise
27 %
28 % STEPS:
29 % 1. Measure total elapsed time since the program started
30 % 2. Measure elapsed time since the last control cycle completed
31 % 3. If elapsed time >= dt: flag a new cycle and reset the per-cycle timer
32 % 4. If elapsed time < dt: flag no cycle this iteration
33 %
34 % =====
35
36 % --- Step 1: Record total elapsed time since program start ---
37 % toc(MGMT.init_time) returns seconds since commsInit() called tic().

```

```
38 % Useful for time-stamping data or implementing timed sequences.
39 MGMT.t_real = toc(MGMT.init_time);
40
41 % --- Step 2: Record time elapsed since the last completed control cycle ---
42 % toc(MGMT.last_call) returns seconds since the last time next_step was true.
43 MGMT.dt_real = toc(MGMT.last_call);
44
45 % --- Steps 3 & 4: Decide whether to trigger a new control cycle ---
46 if MGMT.dt_real > MGMT.dt
47     % Sufficient time has passed – allow the control code to run.
48     MGMT.next_step = true;
49     MGMT.last_call = tic(); % Reset the per-cycle timer for the next interval
50 else
51     % Not enough time has passed yet – skip control execution this iteration.
52     MGMT.next_step = false;
53 end
54
55 end
```


APPENDIX E. Function “svrRead.m”

A direct copy of full MATLAB function. Function responsible for reading all relevant registers from MODBUS server.

```

1  function MGMT = svrRead(MGMT)
2  % =====
3  % svrRead – Read all input registers from the MODBUS server
4  % =====
5  %
6  % PURPOSE:
7  % Reads the latest values of all three input types from the MODBUS
8  % server each control cycle and stores them in the MGMT structure.
9  % The main program then unpacks them into the named variables
10 % DI#, AI# and MI# that are used in the student's control code.
11 %
12 % INPUT / OUTPUT:
13 % MGMT – Management structure. Fields updated:
14 % .dig_in [1x8 double] Digital input states (0 or 1)
15 % .in_reg [1x8 double] Analogue input values (scaled float)
16 % .mi_reg [1x8 double] Move input values from LabVIEW (scaled float)
17 %
18 % REGISTER MAP (addresses relative to panel OFFSET):
19 % Discrete inputs OFFSET + 0 ... OFFSET + 7 – dig_in (8 digital channels)
20 % Input registers OFFSET + 0 ... OFFSET + 7 – in_reg (8 analogue channels)
21 % Holding regs OFFSET + 24 ... OFFSET + 31 – mi_reg (8 LabVIEW-MATLAB channels)
22 %
23 % SCALING NOTE:
24 % MODBUS registers store 16-bit unsigned integers only. Floating-point
25 % values are therefore stored on the server scaled by ×100 and rounded
26 % (e.g. a sensor reading of 1.23 is stored as the integer 123).
27 % Dividing by 100 here restores the original floating-point value.
28 % svrWrite() applies the same ×100 scaling in reverse when writing outputs.
29 %
30 % STEPS:
31 % 1. Read 8 discrete input bits – panel digital inputs (e.g. switches)
32 % 2. Read 8 input registers – panel analogue inputs (e.g. sensors)
33 % and divide by 100 to restore float values
34 % 3. Read 8 holding registers – move inputs written by LabVIEW
35 % and divide by 100 to restore float values
36 %
37 % =====

```

```
38
39 % --- Step 1: Read 8 digital (discrete) inputs ---
40 % MODBUS 'inputs' are discrete input coils – each is a single bit (0 or 1).
41 % Typical uses: limit switches, push-buttons, binary status flags.
42 MGMT.dig_in = read(MGMT.srvr, 'inputs', MGMT.OFFSET, 8);
43
44 % --- Step 2: Read 8 analogue input registers and convert to floats ---
45 % MODBUS 'inputregs' are read-only 16-bit input registers.
46 % These carry analogue measurements from the rig (e.g. position, pressure).
47 % Dividing by 100 reverses the server-side ×100 integer scaling.
48 MGMT.in_reg = read(MGMT.srvr, 'inputregs', MGMT.OFFSET, 8) / 100;
49
50 % --- Step 3: Read 8 move-input holding registers and convert to floats ---
51 % MODBUS 'holdingregs' at OFFSET+24 are written by LabVIEW to send
52 % setpoints, mode flags or other commands to this MATLAB program.
53 % Dividing by 100 reverses the server-side ×100 integer scaling.
54 MGMT.mi_reg = read(MGMT.srvr, 'holdingregs', MGMT.OFFSET + 24, 8) / 100;
55
56 end
```

APPENDIX F. Function “svrWrite.m”

A direct copy of full MATLAB function. Function responsible for writing all relevant registers to MODBUS server.

```

1  function MGMT = svrWrite(MGMT, M00, M01, M02, M03, M04, M05, M06, M07)
2  % =====
3  % svrWrite – Write move-output values to the MODBUS server
4  % =====
5  %
6  % PURPOSE:
7  % Takes the eight output values (M00-M07) calculated by the student's
8  % control code, scales them to integers, and writes them to the MODBUS
9  % holding registers on the server. LabVIEW reads these registers each
10 % cycle and uses data according to user configured LabVIEW program.
11 %
12 % INPUTS:
13 % MGMT – Management structure (.svr, .OFFSET, .mo_reg)
14 % M00-M07 – Eight floating-point output values, one per channel.
15 % These are the commands your control code calculated.
16 %
17 % OUTPUT:
18 % MGMT – Updated structure with .mo_reg holding the scaled integer values
19 % (useful for debugging – inspect MGMT.mo_reg to see what was sent)
20 %
21 % REGISTER MAP:
22 % Holding registers at OFFSET+14 ... OFFSET+21 – 8× move output channels
23 %
24 % SCALING NOTE:
25 % MODBUS registers store 16-bit unsigned integers only. Floating-point
26 % outputs are therefore multiplied by 100 and rounded before writing
27 % (e.g. an output value of 1.23 is written to the register as 123).
28 % svrRead() applies the matching +100 scaling when reading move inputs.
29 %
30 % STEPS:
31 % 1. Scale each output float by ×100 and round to the nearest integer,
32 % storing the results in MGMT.mo_reg
33 % 2. Write all eight scaled values to the server in a single block write
34 % to ensure all outputs reach the server
35 %
36 % =====
37

```

```
38 % --- Step 1: Scale each output value by 100 and round to integer ---
39 % MODBUS registers only hold 16-bit unsigned integers (0-65535).
40 % Multiplying by 100 before rounding preserves two decimal places of
41 % precision in the transmitted value (range ±327.67 before overflow).
42 MGMT.mo_reg(1) = round(M00 * 100);
43 MGMT.mo_reg(2) = round(M01 * 100);
44 MGMT.mo_reg(3) = round(M02 * 100);
45 MGMT.mo_reg(4) = round(M03 * 100);
46 MGMT.mo_reg(5) = round(M04 * 100);
47 MGMT.mo_reg(6) = round(M05 * 100);
48 MGMT.mo_reg(7) = round(M06 * 100);
49 MGMT.mo_reg(8) = round(M07 * 100);
50
51 % --- Step 2: Write all eight values to the server in one block write ---
52 % Sending all outputs in a single write() call is more efficient than eight
53 % separate writes and ensures LabVIEW receives all channel values together
54 % in the same MODBUS transaction.
55 % 'uint16' specifies the MODBUS data type used for the register values.
56 write(MGMT.srvr, 'holdingregs', MGMT.OFFSET + 14, [ ...
57 MGMT.mo_reg(1), ...
58 MGMT.mo_reg(2), ...
59 MGMT.mo_reg(3), ...
60 MGMT.mo_reg(4), ...
61 MGMT.mo_reg(5), ...
62 MGMT.mo_reg(6), ...
63 MGMT.mo_reg(7), ...
64 MGMT.mo_reg(8)], 'uint16');
65
66 end
```



Figure APX 1: Portion of client template main loop responsible for coordination of communication between LabVIEW and 3rd party applications for the purposes of offloading complex control strategies.